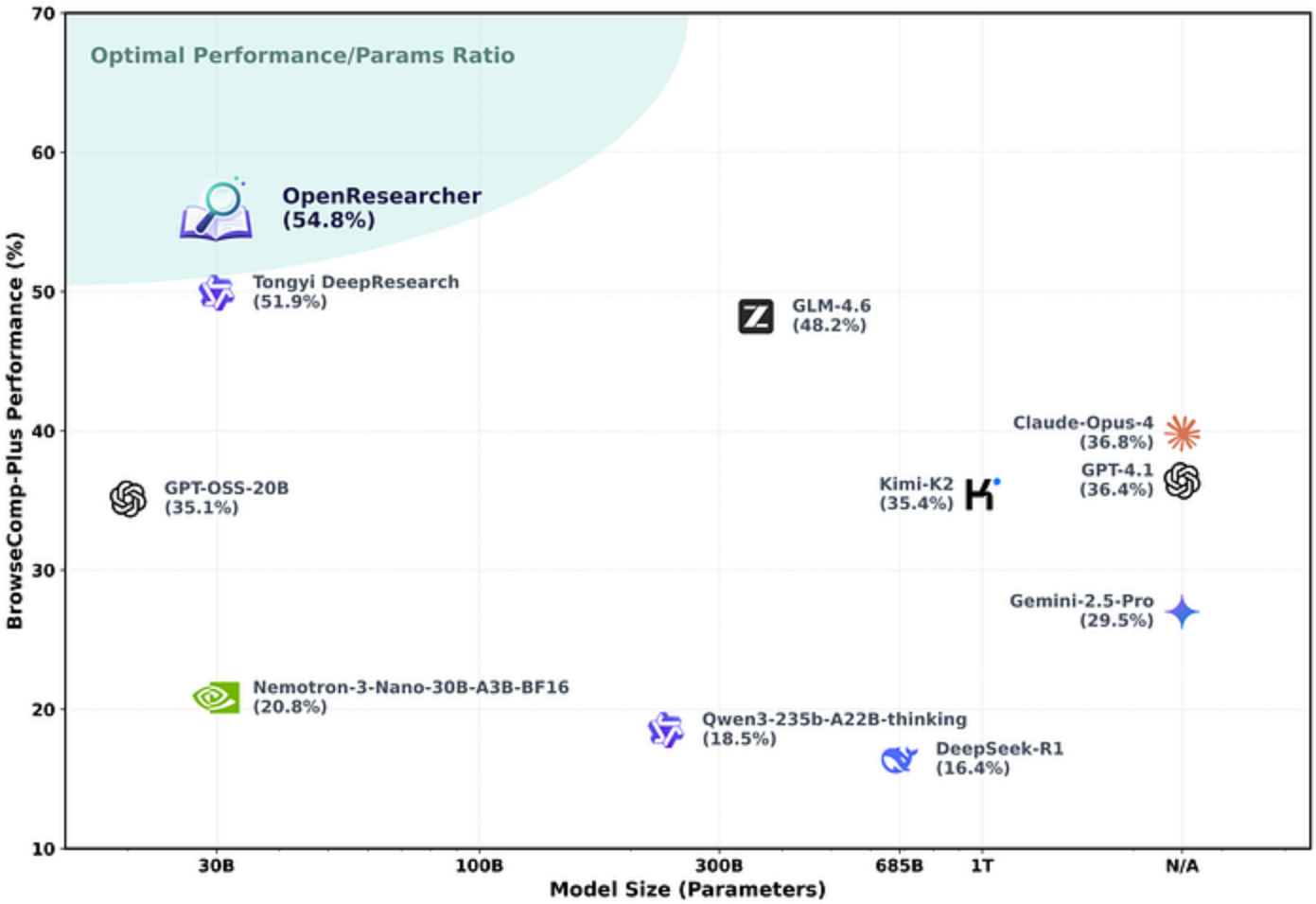
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Deep Research with 96,000+ Trajectories — Completely Offline

Imagine synthesizing human-like research trajectories exceeding 100 turns entirely offline, no reliance on search or scrape APIs, no rate...

Agent Native Follow
~7 min read · February 11, 2026 (Updated: February 11, 2026) · Free: No

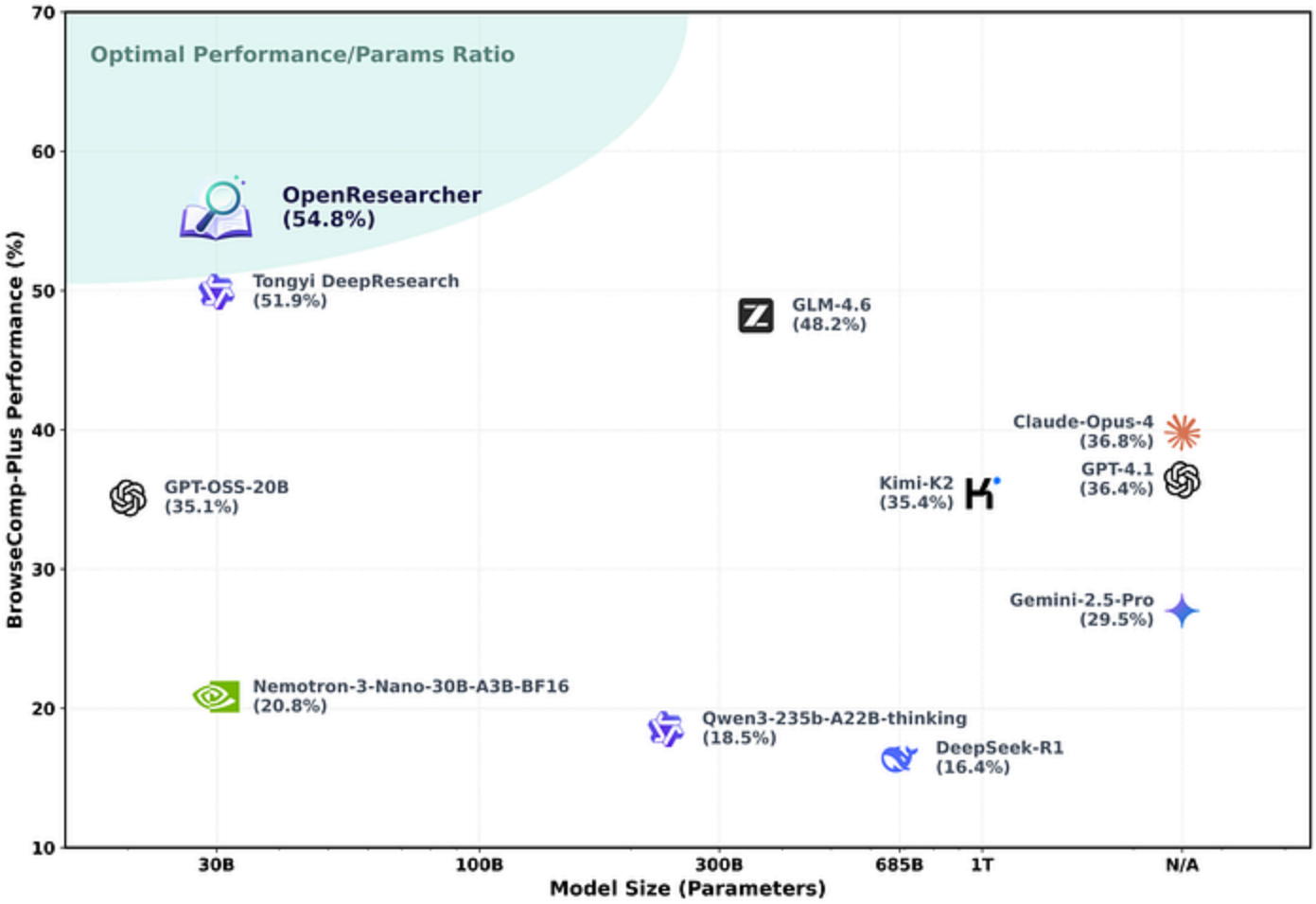
Imagine synthesizing human-like research trajectories exceeding 100 turns entirely offline, no reliance on search or scrape APIs, no rate limits, and crucially, no nondeterminism.

If you could do that, you would get fully reproducible, scalable research workflows without the usual bottlenecks of online services.

That would save you API rate limits, randomness and brittle scrappers.

OpenResearcher solves all that by building a massive offline corpus paired with a local retriever and a powerful 120-billion-parameter GPT model.

This trio enables realistic, long-horizon "browsing-like" tool-use traces that mimic `search > open > find` behaviors you'd expect from a human researcher exploring the web.



This effectively unlocks new possibilities:

- Training models on **deep, multi-step reasoning** over vast knowledge bases
- Avoiding the **unpredictability of live web data** during experiments

- Scaling **research synthesis without API quotas or cost concerns**

OpenResearcher's offline pipeline includes a colossal **10T token corpus** containing **15M curated FineWeb documents** plus **10K gold-standard passages** bootstrapped once for quality and coverage.

It then generates **realistic trajectories filtered by reject sampling to keep only successful, high-quality research paths.**

Fine-tuning on OpenResearcher-generated data boosts the Nemotron-3-Nano-30B-A3B model's accuracy on the **BrowseComp-Plus benchmark from a meager 20.8% to a robust 54.8%, a +34.0 point jump.**

METHODS	BrowseComp-Plus
Foundation Models with Tools	
GPT-4.1	36.4
Claude-4-Opus	36.8
Gemini-2.5-Pro	29.5
Kimi-K2	35.4
DeepSeek-R1	16.4
Nemotron-Nano-30B-A3B	20.8
DeepResearch Agents	
Tongyi DeepResearch	44.5
CutBill-30B-A3B-RL	30.3
Ours	
OpenResearcher-30B-A3B	54.8

METHODS	BrowseComp	GAIA	xbench-DeepSearch
Foundation Models with Tools			
OpenAI o4-mini	28.3	55.8	67.0
Claude-4-Sonnet	12.2	68.3	64.0
Kimi-K2	14.1	57.7	50.0
DeepSeek-V3.1	30.0	63.1	71.0
DeepResearch Agents			
ASearcher-QwQ-32B	5.2	52.8	42.0
WebDancer-QwQ-32B	3.8	51.5	39.0
WebSailor-72B	12.0	55.4	55.0
DeepMiner-32B-SFT	21.2	54.4	53.0
Ours			
OpenResearcher-30B-A3B	30.3	64.1	65.0

In this article, I will break down how OpenResearcher's architecture, trajectory generation, data filtering, and corpus construction work together to deliver these gains and what you can do tomorrow to start building on this foundation.

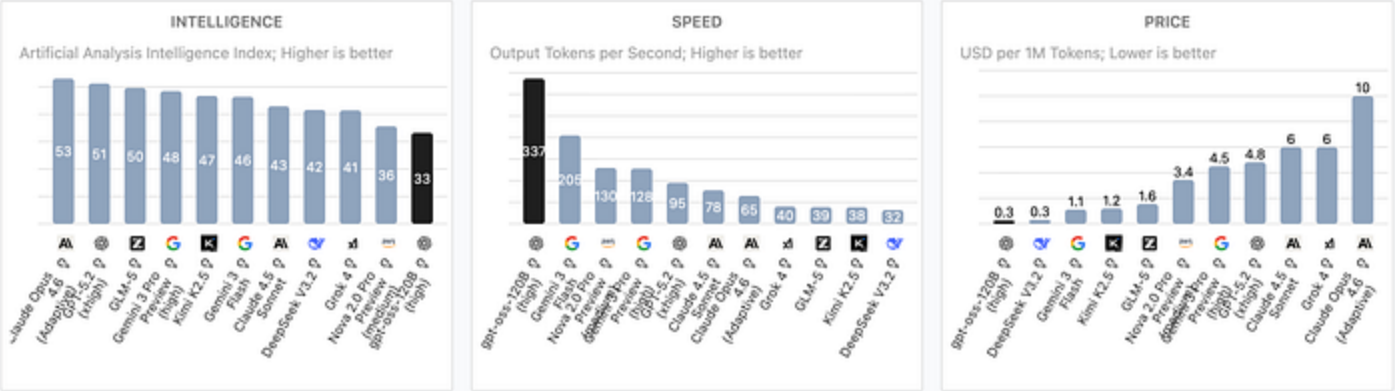
> *Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***Grab the first chapter free:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.*

Core Architecture: GPT-OSS-120B, Local Retriever, and Offline Corpus

OpenResearcher's technical foundation is a deliberate assembly of three heavyweight components designed for scalable, reproducible, and deep research synthesis, and all offline.

GPT-OSS-120B: The Language Backbone

OpenResearcher uses GPT-OSS-120B, a 120-billion-parameter open-source large language model tailored for research tasks.



https://artificialanalysis.ai/models/gpt-oss-120b

Unlike smaller models or black-box APIs, GPT-OSS-120B provides the raw generative power needed to reason over complex, multi-step queries spanning diverse scientific domains.

It's the engine that synthesizes and composes knowledge, but it needs context, this is where retrieval comes in.

Local Retriever: Fast, Deterministic Access to Knowledge

OpenResearcher pairs GPT-OSS-120B with a local retriever system.

This retriever indexes the offline corpus and supports rapid, deterministic lookups during generation.

Instead of querying live web APIs, which are often plagued by rate limits, network latency, and nondeterminism, the retriever operates entirely offline, ensuring repeatable experiments.

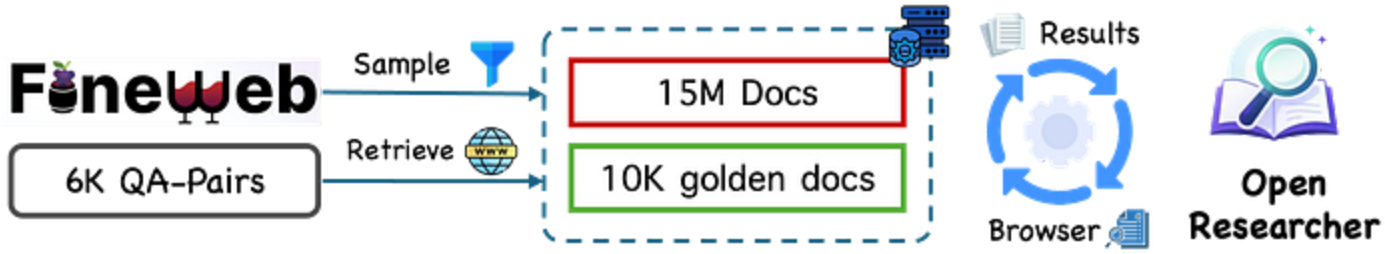
This local retrieval mechanism feeds GPT-OSS-120B the most relevant documents or passages from the corpus before each generation step.

The retriever's design balances precision and recall, enabling the model to **search** the offline knowledge base efficiently and mimic realistic browsing behaviors (`search > open > find`) without ever connecting online.

The 10 Trillion Token Offline Corpus

The offline corpus is the third pillar.

It boasts an immense scale of roughly 10 trillion tokens, comprising 15 million curated FineWeb documents plus 10,000 gold-standard passages.



This corpus is the knowledge reservoir that the retriever taps into. Because it's static and fully offline, it eliminates the unpredictability of live web data, enabling consistent training and evaluation.

The corpus is structured to support deep, multi-turn research trajectories by providing comprehensive coverage and fine-grained evidence passages.

Resources and Tools Provided by OpenResearcher

OpenResearcher is a fully open ecosystem designed for AI builders who want to experiment, iterate, and build on top of a reproducible, offline research synthesis pipeline.

Here's what you get out of the box:

- **Codebase and Pipeline**: The entire OpenResearcher code is available on GitHub, including the logic for trajectory generation, local retrieval, and integration with GPT-OSS-120B.
- **Local Search Engine**: OpenResearcher releases a local retriever engine optimized for fast, deterministic lookups across the massive offline corpus.
- **Corpus Recipe and Dataset**: You get the recipe to build the 10 trillion token offline corpus, including the 15 million FineWeb documents and 10,000 gold-standard passages.
- **Trajectory Datasets**: Over 96,000 realistic, long-horizon research trajectories are released.
- **Evaluation Logs**: OpenResearcher provides comprehensive evaluation logs that document model performance across BrowseComp-Plus and other benchmarks, enabling transparent and reproducible comparisons.
- **Pretrained Models**: Trained checkpoints of Nemotron-3-Nano-30B-A3B fine-tuned on OpenResearcher data are available.
- **Live Demo**: For a quick hands-on experience, OpenResearcher offers a live demo showcasing the pipeline's capabilities in real-time.

Quick Start with OpenResearcher

OpenResearcher team runs the repository with the following setup

- 8 * A100 80G Nvidia GPUs
- Linux operating system

Other hardware setups can also work, but remember to modify the corresponding parameters.

```
sudo apt update
sudo apt install -y openjdk-21-jdk

# install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv --python 3.12
source .venv/bin/activate

# install tevatron for BrowseComp-plus
git clone https://github.com/texttron/tevatron.git
cd tevatron
uv pip install -e .
cd ..

# install all dependencies automatically
uv pip install -e .
```

Setup script then will automatically download BrowseComp-Plus benchmark. Other benchmarks, including BrowseComp, GAIA and xbench-DeepResearch, will be set up automatically when they are first used.

```
bash setup.sh
```

then edit your `.env` file:

```
# Serper API (for web search when using browser_backend=serper)
SERPER_API_KEY=your_key        # Get from: https://serper.dev/

# OpenAI API (for evaluation scoring)
OPENAI_API_KEY=your_key        # Get from: https://platform.openai.com/api-keys
```

You can now deploy your OpenResearcher

```
bash scripts/start_nemotron_servers.sh
```

and run your first task

```
import asyncio
from deploy_agent import run_one, BrowserPool
from utils.openai_generator import OpenAIAsyncGenerator

async def main():
    # Initialize generator and browser
    generator = OpenAIAsyncGenerator(
        base_url="http://localhost:8001/v1",
        model_name="OpenResearcher/OpenResearcher-30B-A3B",
        use_native_tools=True
    )
    browser_pool = BrowserPool(search_url=None, browser_backend="serper")

    # Run deep research
    await run_one(
        question="What is the latest news about OpenAI?",
        qid="quick_start",
        generator=generator,
        browser_pool=browser_pool,
    )

    browser_pool.cleanup("quick_start")

if __name__ == "__main__":
    asyncio.run(main())
```

I want to cover a few more important points to explain how this will change deep research type of workflows moving forward.

No More API Bottlenecks

Imagine a research team building an AI assistant to explore new scientific topics over hundreds of iterative queries.

Traditional approaches choke on API rate limits or changing web content. OpenResearcher's architecture sidesteps this by letting the team run thousands of experiments offline, with full control over the data and retrieval process.

This setup also means you can generate vast datasets of research trajectories to fine-tune smaller models, dramatically improving their performance on benchmarks like BrowseComp-Plus.

OpenResearcher's architecture proves that pairing a massive open LLM with a local retriever and a gargantuan offline corpus is a practical blueprint for building AI research assistants that scale deeply and reproducibly.

Generating Realistic Long-Horizon Browsing-Like Tool-Use Traces

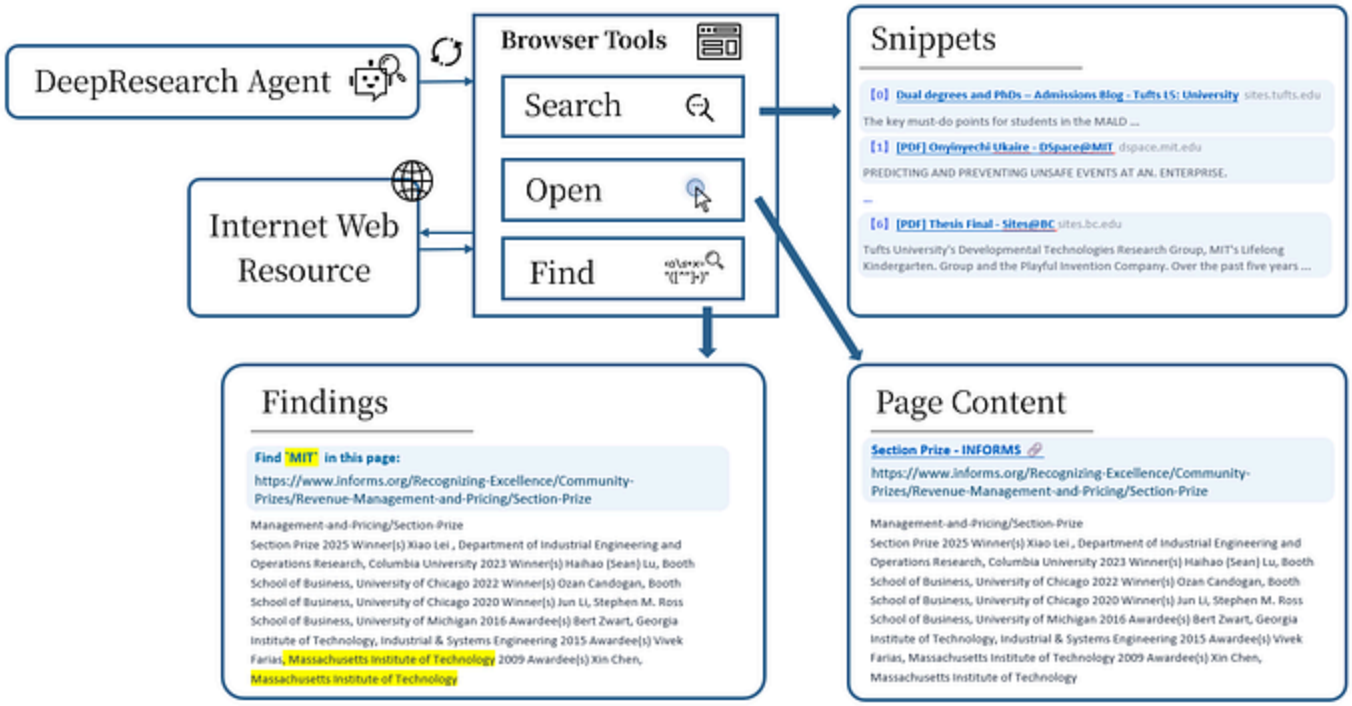
OpenResearcher mimicks human-like browsing trajectory over hundreds of steps because real-world research is iterative and exploratory.

Researchers refine queries, open multiple documents, skim for relevant information, and piece together insights over time.

Capturing this behavior in AI training data is critical to building models that can handle complex, long-horizon tasks.

OpenResearcher explicitly models three core browsing primitives:

- **Search:** Querying the local retriever to find candidate documents relevant to the current research question or sub-question. This mimics typing a query into a search engine.
- **Open:** Selecting and loading a document from the search results to inspect its content. This step simulates clicking on a link or opening a tab.
- **Find:** Extracting or identifying key information or evidence within the opened document that contributes to answering the research question.



This explicit modeling of browsing actions contrasts with simpler "retrieve-and-read" approaches that treat retrieval and reading as a single step.

By breaking down the process, OpenResearcher can generate more realistic and granular trajectories that capture the multi-step nature of research.

How It Works: GPT-OSS-120B + Local Retriever + Corpus

At each step, GPT-OSS-120B generates the next action conditioned on the current trajectory context.

When a **search** action is generated, the local retriever executes the query against the 10 trillion token offline corpus, returning ranked FineWeb documents. The model then decides which document to

open, followed by a **find** action to pinpoint relevant passages.

This pipeline produces a chain of tool-use traces that look and feel like real browsing logs but are entirely offline and reproducible.

The retriever's deterministic nature ensures the same queries yield consistent documents, critical for stable training and evaluation.

You can find further technical details in the official announcement.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests

Benchmarks are the LinkedIn of LLMs. Every model looks unstoppable.

medium.com
#deep-research-agent#deep-research#agentic-search#research#agentic-research

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel